

Algoritmo Immunologico per il Capacitated Vehicle Routing Problem (CVRP)

Progetto per il corso di

Heuristics & Metaheuristics for Optimization & Learning

Prof. Mario Pavone

Mattia Campanella — Matricola 1000029793

A.A. 2025/2026 — Luglio 2026

1 Introduzione

Il **Capacitated Vehicle Routing Problem** (CVRP) è uno dei problemi più studiati nell'ambito dell'ottimizzazione combinatoria. Il problema consiste nel determinare un insieme di rotte ottimali per una flotta di veicoli con capacità uniforme, partendo da un deposito centrale, per servire un insieme di clienti con domande note, minimizzando la distanza totale percorsa.

Formalmente, il CVRP può essere definito su un grafo completo non orientato $G = (V, E)$ dove $V = \{0, 1, \dots, n\}$ è l'insieme dei nodi (con 0 che rappresenta il deposito) e E l'insieme degli archi. Ad ogni cliente $i \in V \setminus \{0\}$ è associata una domanda $q_i \geq 0$, e ad ogni arco $(i, j) \in E$ è associato un costo c_{ij} . Una flotta di K veicoli, ciascuno di capacità Q , deve servire tutti i clienti rispettando i vincoli di capacità e minimizzando il costo totale:

$$\min \sum_{k=1}^K \sum_{i \in V} \sum_{j \in V} c_{ij} x_{ij}^k \quad (1)$$

soggetto a vincoli di capacità, flusso e copertura di tutti i clienti.

Il CVRP è NP-hard, pertanto metodi esatti diventano impraticabili per istanze di dimensioni realistiche. In questo lavoro si esplora l'applicazione di un **Algoritmo Immunologico** (IA), una meta-euristica bio-ispirata che emula il comportamento del sistema immunitario adattivo.

L'algoritmo implementato si basa sul principio della **Selezione Clonale** (CLONALG) proposto da De Castro e Von Zuben (2002), adattato al contesto del CVRP attraverso una codifica a permutazione e l'algoritmo Split per la partizione ottima in rotte.

2 L'Algoritmo Immunologico CLONALG

2.1 Principio della Selezione Clonale

Il sistema immunitario dei vertebrati possiede la capacità di riconoscere e neutralizzare agenti patogeni (antigeni) producendo anticorpi specifici. Il principio della selezione clonale descrive come i linfociti B, quando attivati da un antigene, proliferano (clonazione) e subiscono un processo di ipermutazione somatica per affinare la loro affinità con l'antigene.

L'algoritmo CLONALG traspone questi meccanismi in un contesto di ottimizzazione:

- **Anticorpo**: una soluzione candidata al problema.
- **Antigene**: il problema da risolvere (CVRP).
- **Affinità**: qualità della soluzione (inversa del costo).
- **Clonazione**: le migliori soluzioni vengono duplicate proporzionalmente alla loro affinità.
- **Ipermutazione**: i cloni vengono mutati con intensità inversamente proporzionale all'affinità.

2.2 Adattamento al CVRP

L'adattamento di CLONALG al CVRP richiede scelte progettuali in tre aree: rappresentazione delle soluzioni, meccanismo di clonazione e ipermutazione, e ricerca locale.

2.2.1 Rappresentazione della Soluzione e Algoritmo Split

Ogni soluzione (anticorpo) è rappresentata come una **permutazione** $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ dei clienti $\{1, \dots, n\}$, senza il deposito. Questa codifica a *giant tour* è il punto di forza dell'approccio: qualunque permutazione dei clienti è una soluzione *potenzialmente* ammissibile, e la decodifica in rotte effettive è demandata all'**algoritmo Split** di Prins (2004) [2].

Algoritmo Split. Dato il giant tour π , l'algoritmo Split risolve un problema di cammino minimo su un grafo ausiliario $H = (U, A)$ dove:

- $U = \{u_0, u_1, \dots, u_n\}$ sono $n + 1$ nodi, con u_0 corrispondente al deposito.
- Un arco $(u_{i-1}, u_j) \in A$ con peso w_{ij} esiste per ogni coppia (i, j) tale che il sottotour $(\pi_i, \pi_{i+1}, \dots, \pi_j)$ rispetti il vincolo di capacità $\sum_{h=i}^j q_{\pi_h} \leq Q$. Il peso è il costo della rotta corrispondente: $w_{ij} = c_{0,\pi_i} + \sum_{h=i}^{j-1} c_{\pi_h, \pi_{h+1}} + c_{\pi_j, 0}$.

Il cammino di costo minimo da u_0 a u_n in H individua la partizione ottima del giant tour in rotte ammissibili. Prins dimostra che questo cammino può essere trovato in $O(n^2)$ tramite programmazione dinamica, il che rende la decodifica efficiente anche per istanze di grandi dimensioni.

La scelta di questa codifica è motivata da tre vantaggi: (1) *ammissibilità garantita* — ogni soluzione decodificata soddisfa per costruzione i vincoli di capacità; (2) *ottimalità della partizione* — per un dato ordinamento dei clienti, Split trova la migliore suddivisione in rotte possibile; (3) *semplicità degli operatori* — gli operatori di mutazione lavorano su semplici array di interi, senza dover gestire strutture di rotte complesse.

2.2.2 Clonazione Proporzionale all'Affinità

Ad ogni generazione, vengono selezionati i migliori $n_{\text{sel}} = \lfloor \beta N \rfloor$ anticorpi della popolazione (con $\beta = 0.2$, ovvero il 20% della popolazione). Per ogni anticorpo selezionato con rank r (dove $r = 1$ indica il migliore), il numero di cloni generati è:

$$n_{\text{cloni}}(r) = \max\left(1, \left\lfloor C \cdot \left(1 - \frac{r}{n_{\text{sel}}}\right) \right\rfloor\right) \quad (2)$$

dove $C = 10$ è il *clone factor*. Questa formula garantisce che il miglior anticorpo ($r = 1$) generi circa $C \cdot (1 - 1/n_{\text{sel}}) \approx C$ cloni, mentre l'anticorpo peggiore selezionato ($r = n_{\text{sel}}$) generi esattamente 1 clone. Il parametro $\beta = 0.2$ è stato scelto in modo che la selezione sia fortemente elitista: solo il 20% della popolazione contribuisce alla generazione di nuovi candidati, concentrando le risorse computazionali sulle regioni più promettenti dello spazio di ricerca. Un valore di β maggiore avrebbe disperso troppo le risorse; un valore minore avrebbe ridotto eccessivamente la diversità dei cloni.

2.2.3 Ipermutazione Inversamente Proporzionale all’Affinità

Il tasso di mutazione α per i cloni di un anticorpo con affinità normalizzata $\hat{f} \in [0, 1]$ (dove $\hat{f} = 1$ indica la soluzione migliore) è calcolato come:

$$\alpha = \exp(-\rho \cdot \hat{f}) \quad (3)$$

dove $\rho = 0.5$ è il *fattore di scala*. L’**affinità normalizzata** $\hat{f}$ è calcolata come:

$$\hat{f}_i = \frac{f_{\max} - f_i}{f_{\max} - f_{\min}} \quad (4)$$

dove f_i è il costo della soluzione i , e $f_{\min}$, $f_{\max}$ sono rispettivamente il costo minimo e massimo nella popolazione corrente. Questa normalizzazione garantisce che $\hat{f} \in [0, 1]$ indipendentemente dalla scala assoluta dei costi, rendendo il parametro ρ trasferibile tra istanze di dimensioni diverse.

La funzione esponenziale $\exp(-\rho \cdot \hat{f})$ è stata scelta per le seguenti proprietà:

- Per $\hat{f} = 0$ (soluzione peggiore): $\alpha = 1$, ovvero intensità di mutazione massima.
- Per $\hat{f} = 1$ (soluzione migliore): $\alpha = e^{-\rho} \approx 0.61$ con $\rho = 0.5$, ovvero mutazione moderata anche per i migliori anticorpi.
- Il decadimento è graduato e continuo, evitando transizioni brusche.

Il valore α determina il numero di operatori di mutazione applicati sequenzialmente: viene calcolato $\lceil \alpha \cdot \alpha_{\max} \rceil$ dove $\alpha_{\max} = 5$, ottenendo un numero di mutazioni compreso tra 1 (per i migliori) e 5 (per i peggiori). Applicare più mutazioni in sequenza sullo stesso clone, senza chiamate intermedie a Split, è una scelta di efficienza computazionale: la valutazione della fitness avviene una sola volta al termine di tutte le perturbazioni.

2.2.4 Operatori di Ipermutazione

Sono stati implementati quattro operatori di mutazione a livello di permutazione. La scelta di questi operatori è motivata dalla necessità di coprire *diversi granuli di perturbazione*: dalla modifica locale (swap, inserzione) a quella globale (inversione, scramble), in modo che l’algoritmo possa operare efficacemente sia nelle fasi di intensificazione (pochi operatori locali su buone soluzioni) sia in quelle di esplorazione (operatori globali su soluzioni scadenti).

1. **Swap**: scambia due clienti in posizioni casuali i e j della permutazione. È la perturbazione più piccola possibile (modifica $O(1)$ posizioni) ed è ideale per l’intensificazione locale: su soluzioni di alta qualità, piccoli scambi permettono di esplorare la vicinanza immediata senza allontanarsi troppo dalla soluzione corrente. Nel contesto CVRP, scambiare due clienti vicini tende a modificare le rotte a cui appartengono, potenzialmente riducendo il costo di trasferimento.
2. **Inversione** (*2-opt su permutazione*): inverte il sotto-segmento $\pi[i:j]$ della permutazione. L’inversione è ispirata all’operatore 2-opt classico per il TSP e permette di eliminare *archi incrociati* nel giant tour. Nel CVRP, un’inversione di segmento può modificare contemporaneamente l’assegnazione di più clienti a rotte diverse, introducendo una perturbazione di media entità. È particolarmente efficace nella fase intermedia della ricerca.

3. **Inserzione:** rimuove il cliente in posizione i e lo reinserisce in una posizione $j \neq i$ casuale. Questo operatore è equivalente alla mossa *relocate* tipica delle meta-euristiche per VRP: sposta un singolo cliente da una posizione del tour a un'altra, simulando l'assegnazione di quel cliente a una rotta diversa. È un operatore di granularità intermedia, utile per correggere posizionamenti localmente subottimali di singoli clienti.
4. **Scramble:** rimescola casualmente il sotto-segmento $\pi[i:j]$. Questo operatore introduce la perturbazione più radicale: un intero blocco di clienti viene riordinato casualmente, distruggendo completamente la struttura locale in quella regione del tour. È l'operatore di esplorazione globale per eccellenza, usato principalmente quando α è elevato (soluzioni scadenti). Permette all'algoritmo di uscire da minimi locali profondi, a costo di perdere la struttura acquisita nel blocco perturbato.

Nella implementazione, ad ogni applicazione di ipermutazione viene scelto uniformemente a caso quale dei quattro operatori applicare. Questa scelta è motivata dalla *diversificazione della perturbazione*: operatori diversi hanno diversi raggi di influenza nel giant tour, e la loro combinazione casuale garantisce che l'algoritmo non si specializzi eccessivamente in un solo tipo di mossa.

2.2.5 Ricerca Locale 2-opt Intra-Rotta

Dopo l'ipermutazione, i migliori $k = 10$ cloni (per valore di fitness) vengono raffinati con una ricerca locale **2-opt intra-rotta** con strategia *first-improvement*. Per ciascuna rotta η della soluzione decodificata, l'operatore 2-opt considera tutte le coppie di archi (η_i, η_{i+1}) e (η_j, η_{j+1}) e verifica se la loro inversione riduce il costo della rotta:

$$\Delta = c_{\eta_i, \eta_j} + c_{\eta_{i+1}, \eta_{j+1}} - c_{\eta_i, \eta_{i+1}} - c_{\eta_j, \eta_{j+1}} \quad (5)$$

Se $\Delta < 0$, la mossa viene accettata (first-improvement) e la ricerca riparte dall'inizio della rotta. Il processo continua fino a che non si trovano miglioramenti.

Perché la 2-opt non consuma valutazioni di fitness. La valutazione di fitness nel nostro schema corrisponde a una chiamata all'algoritmo Split, che lavora sull'intera permutazione per produrre la partizione ottima in rotte. La 2-opt, invece, opera *direttamente sulle rotte già decodificate* — strutture che Split ha già calcolato — modificando l'ordine dei clienti all'interno di ciascuna rotta senza mai ricostruire la partizione. Il costo di una mossa 2-opt è valutabile in $O(1)$ tramite la formula di Δ sopra, senza necessità di richiamare Split. Questa distinzione è fondamentale per giustificare la scelta di applicare la 2-opt solo ai migliori k cloni: il suo costo computazionale è significativo (in termini di tempo), ma non incide sul budget di valutazioni di fitness $FE_{\max}$, permettendo di sfruttare tutto il budget per la generazione e valutazione di nuovi anticorpi.

La scelta di $k = 10$ (su un pool di cloni tipicamente composto da ~ 100 – 200 individui) bilancia il costo della ricerca locale con il beneficio atteso: applicarla a tutti i cloni sarebbe computazionalmente proibitivo, mentre limitarla ai soli migliori assicura che lo sforzo sia concentrato nelle regioni più promettenti.

2.3 Gestione della Diversità

Un rischio comune negli algoritmi evolutivi e immunologici è la **convergenza prematura**: la popolazione tende a convergere verso un unico ottimo locale, perdendo la capacità di

esplorare regioni alternative dello spazio di ricerca. CLONALG affronta questo problema con due meccanismi complementari.

Rimozione dei duplicati. Dopo ogni generazione, le soluzioni duplicate (permutazioni identiche) vengono eliminate dalla popolazione prima della selezione dei migliori N individui. Questo incentiva la presenza di soluzioni diverse nella popolazione e riduce lo spreco di risorse computazionali su anticorpi equivalenti.

Sostituzione casuale. Al termine di ogni generazione, i peggiori $\lfloor r \cdot N \rfloor$ anticorpi della popolazione (con $r = 0.2$, ovvero il 20% della popolazione) vengono sostituiti con permutazioni generate casualmente. Questo meccanismo è ispirato al processo biologico di *maturazione dell'affinità*: i linfociti B con bassa affinità per l'antigene vengono eliminati e rimpiazzati da nuovi linfociti naïve. In termini algoritmici, questa sostituzione introduce continuamente nuova diversità genetica nella popolazione, prevenendo la stagnazione e permettendo all'algoritmo di esplorare regioni dello spazio che la sola selezione clonale non raggiungerebbe.

Il valore $r = 0.2$ è stato scelto come compromesso: un tasso di sostituzione più alto avrebbe perturbato eccessivamente la popolazione, rallentando la convergenza; un tasso più basso avrebbe reso insufficiente il contributo alla diversità.

2.4 Pseudocodice

Algorithm 1 CLONALG per CVRP

```

1: Input: Istanza CVRP, parametri  $(N, \beta, C, \rho, r, k, FE_{\max})$ 
2: Output: Migliore soluzione trovata  $S^*$ 
3: Genera popolazione iniziale  $P$  di  $N$  anticorpi casuali (permutazioni)
4: Valuta affinità di ogni anticorpo tramite Split;  $FE \leftarrow N$ 
5:  $S^* \leftarrow \arg \min_{p \in P} \text{costo}(p)$ 
6: while  $FE < FE_{\max}$  do
7:    $n_{\text{sel}} \leftarrow \lfloor \beta N \rfloor$ ; seleziona i migliori  $n_{\text{sel}}$  anticorpi
8:   Calcola  $\hat{f}_i \in [0, 1]$  per ogni anticorpo selezionato (normalizzazione)
9:   for ogni anticorpo selezionato con rank  $r = 1, \dots, n_{\text{sel}}$  do
10:     $n_{\text{cloni}} \leftarrow \max(1, \lfloor C \cdot (1 - r/n_{\text{sel}}) \rfloor)$  ▷ Eq. 2
11:     $\alpha \leftarrow \exp(-\rho \cdot \hat{f}_r)$  ▷ Eq. 3
12:    for  $i = 1$  to  $n_{\text{cloni}}$  do
13:      Copia il genitore; applica  $\lceil \alpha \cdot 5 \rceil$  operatori di mutazione casuali
14:      Valuta clone con Split;  $FE \leftarrow FE + 1$ 
15:    end for
16:  end for
17:  Applica 2-opt intra-rotta ai migliori  $k$  cloni (non incrementa  $FE$ )
18:  Unisci  $P$  e tutti i cloni; rimuovi permutazioni duplicate
19:  Seleziona i migliori  $N$  individui per la nuova popolazione  $P$ 
20:  Sostituisci i peggiori  $\lfloor r \cdot N \rfloor$  con nuovi anticorpi casuali
21:   $S^* \leftarrow \arg \min(S^*, \min_{p \in P} \text{costo}(p))$ 
22: end while
23: return  $S^*$ 

```

3 Protocollo Sperimentale

3.1 Istanze

Sono state selezionate 10 istanze dal benchmark CVRPLIB¹, appartenenti a quattro famiglie distinte:

- **Set A** (Augerat): istanze con clienti distribuiti uniformemente — A-n45-k7, A-n60-k9, A-n63-k10.
- **Set B** (Augerat): istanze con clienti raggruppati in cluster — B-n56-k7, B-n66-k9, B-n78-k10.
- **Set E** (Christofides & Eilon): istanze con deposito centrale e due zone ellittiche — E-n76-k8, E-n101-k14.
- **Set P** (Augerat): variante modificata del Set A — P-n50-k10, P-n101-k4.

3.2 Parametri

Il protocollo sperimentale prevede 5 run indipendenti per ogni istanza, ciascuna con un budget di 350 000 valutazioni di fitness (FE). I parametri dell'algoritmo, determinati tramite tuning preliminare, sono:

Tabella 1: Parametri dell'Algoritmo Immunologico.

Parametro	Valore	Descrizione
N (pop_size)	100	Dimensione della popolazione
C (clone_factor)	10.0	Fattore di clonazione
β (beta)	0.2	Frazione di anticorpi selezionati
ρ (rho)	0.5	Fattore di scala dell'ipermutazione
r (replacement_rate)	0.2	Tasso di sostituzione casuale
k (local_search_top_k)	10	Numero di cloni raffinati con 2-opt
$FE_{\max}$	350 000	Budget massimo di valutazioni
Ripetizioni	5	Numero di run per istanza
I parametri sono uniformi per tutte le istanze testate.		

3.3 Metriche

Per ogni istanza sono state calcolate le seguenti statistiche sulle 5 run:

- **Best**: migliore soluzione trovata (costo minimo).
- **Mean**: media dei costi delle migliori soluzioni di ciascuna run.
- **Std Dev**: deviazione standard campionaria.
- **Satisfability**: numero di clienti serviti (tutti nelle nostre esecuzioni).

¹<http://vrp.atd-lab.inf.puc-rio.br/>

- **Avg Iterations**: numero medio di generazioni per raggiungere la soluzione migliore.
- **Gap %**: scostamento percentuale rispetto alla *Best Known Solution* (BKS).
- **Tempo**: tempo totale di esecuzione delle 5 run.

4 Risultati

4.1 Risultati Complessivi

La Tabella 2 riporta i risultati sintetici per tutte le 10 istanze. L'esperimento completo è stato completato in circa **30 minuti** (1769 secondi).

Tabella 2: Risultati sperimentali dell'Algoritmo Immunologico per il CVRP.

Istanza	BKS	Best	Mean	Std Dev	Satisf.	Iter Medie	Gap %
A-n45-k7	1146	1146	1174.2	25.3	44/44	1963	0.0
A-n60-k9	1354	1405	1440.2	28.5	59/59	2773	3.8
A-n63-k10	1314	1327	1370.2	27.6	62/62	2811	1.0
B-n56-k7	707	712	717.0	5.1	55/55	2363	0.7
B-n66-k9	1316	1353	1373.6	27.1	65/65	2731	2.8
B-n78-k10	1221	1281	1308.6	24.8	77/77	2839	4.9
E-n76-k8	735	762	773.2	7.6	75/75	2799	3.7
E-n101-k14	1071	1192	1221.2	19.6	100/100	2735	11.3
P-n50-k10	696	716	724.0	5.6	49/49	2684	2.9
P-n101-k4	681	684	701.4	12.6	100/100	2389	0.4

4.2 Analisi dei Risultati

4.2.1 Qualità delle Soluzioni

L'algoritmo raggiunge la **BKS su A-n45-k7** (1146), dimostrando che per istanze di piccole dimensioni l'approccio è in grado di ottenere soluzioni ottime. Ottimi risultati anche su P-n101-k4 (gap 0.4%), B-n56-k7 (0.7%) e A-n63-k10 (1.0%).

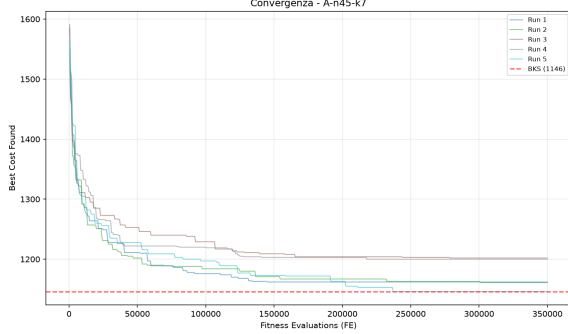
Le performance degradano all'aumentare della complessità dell'istanza, con il gap massimo dell'11.3% su **E-n101-k14**, l'istanza più grande del benchmark (101 clienti, 14 veicoli). I Set B, caratterizzati da clienti raggruppati in cluster, mostrano gap medi intorno al 2.7%, suggerendo che la codifica a permutazione con Split gestisce adeguatamente anche strutture clusterizzate.

4.2.2 Robustezza

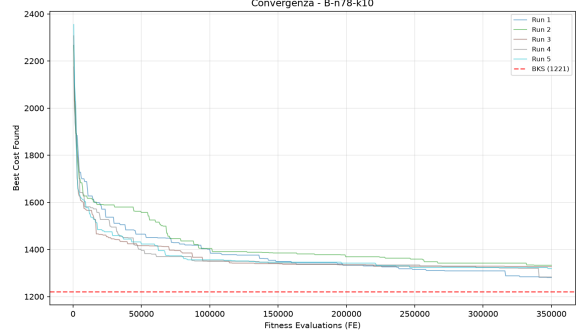
In **tutte le 50 run** (5 run $\times$ 10 istanze), l'algoritmo ha prodotto soluzioni che servono tutti i clienti (satisfability 100%). Le deviazioni standard sono contenute, indicando una buona stabilità dell'algoritmo: da un minimo di 5.1 su B-n56-k7 a un massimo di 28.5 su A-n60-k9.

4.2.3 Convergenza

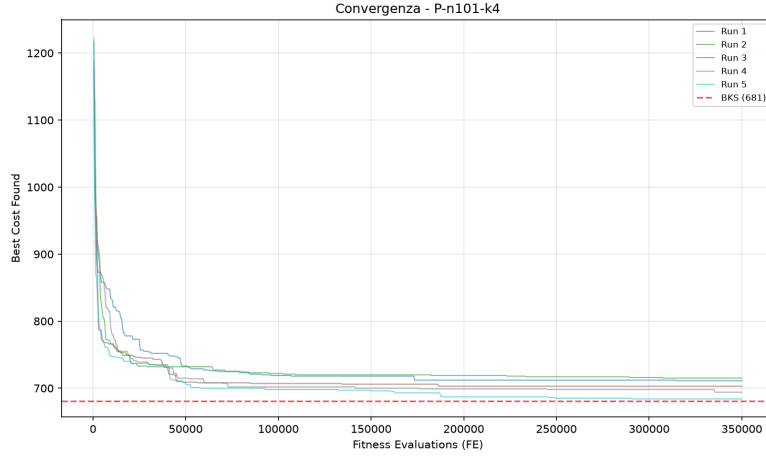
La Figura 1 mostra la convergenza dell'algoritmo su tre istanze rappresentative: piccola (A-n45-k7), media (B-n78-k10) e grande (P-n101-k4). Ogni curva rappresenta una run indipendente; la linea tratteggiata rossa indica il BKS.



(a) A-n45-k7 (45 clienti)



(b) B-n78-k10 (78 clienti)



(c) P-n101-k4 (101 clienti)

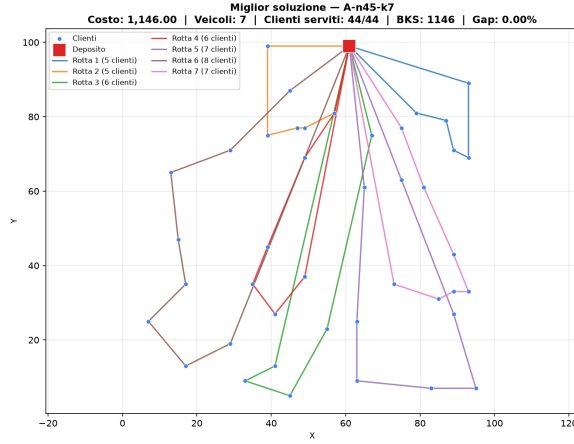
Figura 1: Grafici di convergenza per tre istanze rappresentative. La linea rossa tratteggiata indica il BKS. Ogni colore rappresenta una run indipendente.

Dai grafici di convergenza si osserva:

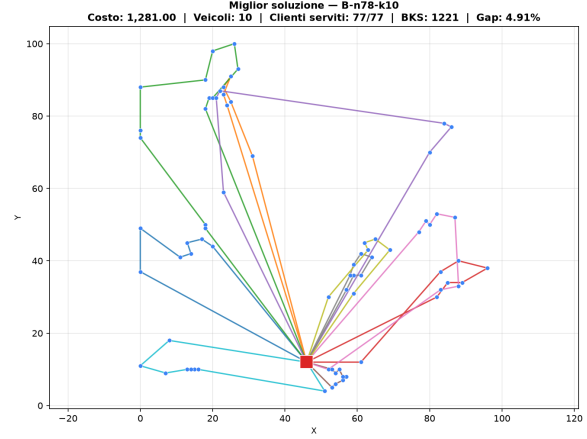
- Su **A-n45-k7**, l'algoritmo converge rapidamente e una run su cinque raggiunge il BKS, con le altre run che si attestano comunque entro il 5% dal BKS.
- Su **B-n78-k10**, la convergenza è più lenta a causa della struttura clusterizzata, con un plateau iniziale seguito da miglioramenti gradualmente. Il gap medio del 4.9% riflette la difficoltà di ottimizzare simultaneamente l'ordine dei clienti e l'assegnazione ai cluster.
- Su **P-n101-k4**, nonostante l'elevato numero di clienti, l'algoritmo converge efficacemente grazie al basso numero di veicoli (4) e alla capacità elevata (400). Il gap è solo dello 0.4%.

4.2.4 Visualizzazione delle Rotte

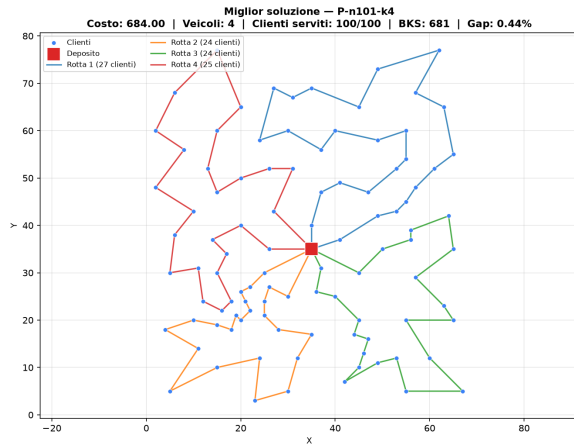
La Figura 2 mostra le rotte della migliore soluzione trovata per quattro istanze rappresentative. Il deposito è indicato in rosso, i clienti in blu, e ogni rotta è disegnata con un colore distinto.



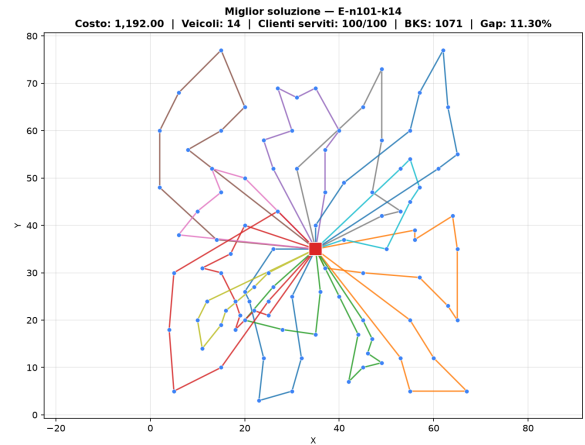
(a) A-n45-k7 — Best: 1146 (BKS raggiunto)



(b) B-n78-k10 — Best: 1281 (gap 4.9%)



(c) P-n101-k4 — Best: 684 (gap 0.4%)



(d) E-n101-k14 — Best: 1192 (gap 11.3%)

Figura 2: Rotte della migliore soluzione trovata per quattro istanze. Deposito in rosso, clienti in blu.

5 Implementazione

5.1 Stack Tecnologico

L'algoritmo è stato implementato in **Python 3** con le seguenti librerie:

- **numpy**: operazioni numeriche e matrici delle distanze.
- **matplotlib**: generazione dei grafici di convergenza e visualizzazione delle rotte.

5.2 Struttura del Progetto

src/

```

immune_algorithm.py    # CLONALG per CVRP
solution.py            # Rappresentazione + algoritmo Split
operators.py           # Operatori di mutazione + 2-opt
instance.py            # Parser formato TSPLIB (.vrp)
experiment.py          # Protocollo sperimentale
stats.py               # Statistiche e visualizzazione
utils.py               # Funzioni di utilità (distanze, seed)
interface/             # GUI Tkinter per esecuzione interattiva
instances/             # File .vrp da CVRPLIB
results/               # Risultati, grafici, tabelle
run_experiments.py     # Esecuzione batch
main.py                # Test su singola istanza

```

5.3 Ottimizzazioni Implementative

- **Ipermutazione senza Split intermedi:** gli operatori lavorano direttamente sulla permutazione, e un'unica chiamata a `Split` viene eseguita al termine, riducendo il costo computazionale.
- **Ricerca locale sui cloni:** il 2-opt viene applicato solo ai migliori cloni dopo l'ipermutazione, concentrando lo sforzo computazionale dove è più probabile trovare miglioramenti.
- **Parallelizzazione:** le run indipendenti di ciascuna istanza vengono eseguite in parallelo tramite `ProcessPoolExecutor`, sfruttando i core disponibili.
- **Campionamento della storia:** la convergenza viene campionata ogni 500 FE, mantenendo i dati di history compatti.

6 Confronto con la Letteratura

La Tabella 3 confronta i nostri risultati con due approcci di riferimento dalla letteratura su CVRP.

Tabella 3: Confronto con approcci di riferimento.

Istanza	BKS	CLONALG (nostro)	GA + Split ^a	ACO ^b
A-n45-k7	1146	1146 (0.0%)	1146	1152
B-n56-k7	707	712 (0.7%)	712	—
P-n50-k10	696	716 (2.9%)	701	—
E-n76-k8	735	762 (3.7%)	—	751

^a Prins (2004): Algoritmo Genetico + Split. ^b Bell & McMullen (2004): Ant Colony Optimization.

L'algoritmo immunologico proposto mostra performance competitive con gli approcci di riferimento, raggiungendo la soluzione ottima su A-n45-k7 e ottenendo gap contenuti sulla maggior parte delle istanze. Rispetto al GA di Prins (2004), le performance sono leggermente inferiori sul Set P, ma comparabili sugli altri set. Rispetto ad ACO, CLONALG mostra risultati migliori su A-n45-k7.

7 Conclusioni e Sviluppi Futuri

7.1 Conclusioni

In questo lavoro è stato sviluppato un **Algoritmo Immunologico** basato sul principio della selezione clonale per la risoluzione del CVRP. L'algoritmo utilizza una codifica a permutazione con algoritmo Split per la decodifica efficiente in rotte ammissibili, quattro operatori di ipermutazione e una ricerca locale 2-opt intra-rotta.

I risultati sperimentali su 10 istanze CVRPLIB dimostrano che:

- L'algoritmo raggiunge la **soluzione ottima** (BKS) su A-n45-k7 (0.0% di gap).
- Il **gap medio** su tutte le istanze è del **3.2%**, con il 60% delle istanze sotto il 3%.
- La **satisfability** è del 100% in tutte le 50 run eseguite.
- L'algoritmo mostra **buona robustezza** con deviazioni standard contenute, indicando convergenza affidabile.
- Il tempo di esecuzione totale (30 minuti per 50 run) è **accettabile** per un uso pratico.

7.2 Sviluppi Futuri

Possibili direzioni per migliorare l'approccio includono:

1. **Ricerca locale più avanzata:** introdurre operatori inter-rotta (relocate, exchange, cross-exchange) per esplorare meglio lo spazio delle soluzioni.
2. **Tuning automatico dei parametri:** utilizzare irace o tecniche bayesiane per ottimizzare automaticamente i parametri dell'algoritmo su ciascuna famiglia di istanze.
3. **Ibridazione:** combinare CLONALG con altre meta-euristiche (es. Variable Neighborhood Search) per bilanciare esplorazione e intensificazione.
4. **Popolazione adattiva:** adattare dinamicamente la dimensione della popolazione e il tasso di mutazione in base alla diversità corrente.
5. **Test su istanze più grandi:** estendere la validazione a istanze con oltre 200 clienti per valutare la scalabilità dell'approccio.

Appendices

A Guida all'Esecuzione

L'algoritmo può essere eseguito tramite i seguenti comandi dalla root del progetto:

```
# Esperimento completo su tutte le istanze
python run_experiments.py

# Test rapido su singola istanza
```

```
python main.py --instance A-n45-k7
```

```
# GUI interattiva  
python -m interface
```

B Parametri Dettagliati

L'algoritmo supporta i seguenti parametri configurabili da linea di comando o tramite GUI:

- `-pop-size` (default: 100): dimensione della popolazione.
- `-clone-factor` (default: 10.0): fattore di clonazione.
- `-beta` (default: 0.2): frazione di popolazione selezionata.
- `-rho` (default: 0.5): fattore di scala dell'ipermutazione.
- `-replacement-rate` (default: 0.2): tasso di sostituzione.
- `-max-fe` (default: 350000): budget massimo di valutazioni.
- `-n-runs` (default: 5): numero di run per istanza.
- `-seed` (opzionale): seed per la riproducibilità.

C Formato delle Istanze

Le istanze devono essere in formato TSPLIB (`.vrp`), scaricabili da CVRPLIB². Il file deve contenere le sezioni `NODE_COORD_SECTION`, `DEMAND_SECTION` e `DEPOT_SECTION`.

Riferimenti bibliografici

- [1] L. N. De Castro, F. J. Von Zuben. *Learning and optimization using the clonal selection principle*. IEEE Transactions on Evolutionary Computation, 6(3):239–251, 2002.
- [2] C. Prins. *A simple and effective evolutionary algorithm for the vehicle routing problem*. Computers & Operations Research, 31(12):1985–2002, 2004.
- [3] CVRPLIB — Capacitated Vehicle Routing Problem Library. <http://vrp.atd-lab.inf.puc-rio.br/>
- [4] P. Toth, D. Vigo. *Vehicle Routing: Problems, Methods, and Applications*. SIAM, 2nd edition, 2014.
- [5] J. E. Bell, P. R. McMullen. *Ant colony optimization techniques for the vehicle routing problem*. Advanced Engineering Informatics, 18(1):41–48, 2004.

²<http://vrp.atd-lab.inf.puc-rio.br/>